

Simulare de Curse Auto 3D în Unity: Fizică Realistă a Vehiculelor și Inteligență Artificială pentru Oponenți

Vasile Daria-Gabriela
daria-gabriela.vasile@s.unibuc.ro

Turcu Filip-Ianis
filip-ianis.turcu@s.unibuc.ro

Borozan George
george.borozan@s.unibuc.ro

Pitica Nicola
nicola.pitica@s.unibuc.ro

Gheorghies Petruț-Rareș
petrut-ares.gheorghies@s.unibuc.ro

Voicu Ioan-Vladut
vvlad424@gmail.com

Popescu Tiberiu-Andrei
tiberiu-andrei.popescu@s.unibuc.ro

Rezumat

Lucrarea descrie proiectarea și implementarea unui joc de curse auto 3D în motorul de jocuri Unity (versiunea 6000.4), cu accent pe simularea realistă a fizicii vehiculelor și pe inteligența artificială a oponenților. Proiectul abordează trei provocări fundamentale din domeniul dezvoltării jocurilor: (1) modelarea comportamentului dinamic al unui vehicul folosind sistemul WheelCollider din Unity, cu suport pentru tracțiune integrală (AWD), control al tracțiunii și asistență laterală la aderență; (2) implementarea unui sistem de navigare bazat pe waypoint-uri pentru vehiculele AI, cu adaptare dinamică a vitezei prin zone de boost și încetinire; și (3) definirea traseului de curse printr-un sistem de waypoint-uri configurabile manual în editorul Unity. Rezultatele demonstrează un model de conducere stabil și responsiv, oponenți AI capabili să navigheze autonom pe traseu și un sistem complet de gestionare a curselor cu numărătoare inversă, tururi și ecran de final.

1 Introducere

Jocurile de curse auto reprezintă unul dintre cele mai populare genuri din industria jocurilor video, combinând elemente de fizică, grafică 3D și inteligență artificială într-o experiență interactivă complexă [1]. Simularea realistă a dinamicii vehiculelor constituie

o provocare inginerească semnificativă, deoarece necesită modelarea interacțiunii dintre roți, suspensie, motor și suprafața drumului [2].

Proiectul prezentat în această lucrare constă în dezvoltarea unui joc de curse auto 3D utilizând motorul Unity (versiunea 6000.4.2f1), cu scopul de a demonstra aplicarea practică a conceptelor din domeniul Proiectării Avansate a Sistemelor (PAS). Obiectivele principale ale proiectului sunt:

- **Fizica vehiculului jucătorului:** Implementarea unui model de conducere realist bazat pe componenta WheelCollider din Unity, cu suport pentru tracțiune integrală, frânare, coasting (rulare liberă), control al tracțiunii și asistență la aderență laterală.
- **Inteligența artificială a oponenților:** Proiectarea unui sistem de navigare bazat pe waypoint-uri (puncte de referință), care permite vehiculelor controlate de calculator să parcurgă autonom traseul de curse, cu adaptare dinamică a vitezei.
- **Definirea traseului prin waypoint-uri:** Construcția traseului de curse folosind un sistem de waypoint-uri configurabile direct în editorul Unity, care servește atât pentru navigarea AI, cât și pentru delimitarea traiectoriei circuitului.
- **Sisteme de joc:** Implementarea sistemelor auxiliare precum numărătoare inversă la start, con-

torizarea tururilor, meniul de pauză și ecranul de sfârșit al cursei.

Lucrarea este organizată după cum urmează: Secțiunea 2 prezintă lucrări conexe, Secțiunea 3 detaliază arhitectura și implementarea, Secțiunea 4 prezintă rezultatele obținute, iar Secțiunile 5 și 6 discută direcții viitoare și concluzii.

2 Lucrări Conexe

2.1 Simularea Fizicii Vehiculelor

Modelarea dinamicii vehiculelor în jocurile video a fost un subiect de cercetare activ de-a lungul deceniilor. Modelul Pacejka (cunoscut și sub denumirea de “Magic Formula”) [2] oferă o aproximare matematică a forțelor generate de anvelope în funcție de alunecare, fiind utilizat extensiv în simulatoarele de curse profesionale. Unity oferă componenta `WheelCollider`, care implementează un model simplificat de suspensie și fricțiune bazat pe curbe configurabile de aderență (stiffness), permițând un compromis între realism și performanță [3].

Abordarea noastră extinde funcționalitatea de bază a `WheelCollider`-ului prin adăugarea unor sisteme suplimentare: controlul tracțiunii (traction control), asistența la aderența laterală (lateral grip assist), downforce simulat și un model de coasting care înlocuiește frânarea bruscă cu drag aerodinamic gradual.

2.2 Inteligența Artificială în Jocurile de Curse

Navigarea vehiculelor AI în jocuri de curse este abordată frecvent prin sisteme bazate pe waypoint-uri [1], unde o secvență de puncte de referință definește traseul pe care vehiculul trebuie să îl urmeze. Alternativ, abordări mai avansate utilizează câmpuri de potențial, rețele neuronale sau algoritmi de învățare prin recompensă (reinforcement learning) [5].

Proiectul nostru adoptă abordarea clasică bazată pe waypoint-uri, cu adăugarea unor elemente de variabilitate: fiecare waypoint are o lățime configurabilă, iar poziția țintă este aleasă aleatoriu în cadrul acestei lățimi, ceea ce introduce un comportament mai natural al oponentilor.

2.3 Definirea Traseelor prin Waypoint-uri

Definirea traseelor de curse în jocuri video se poate realiza prin mai multe abordări: modelare manuală a geometriei, generare procedurală [4] sau utilizarea unor sisteme de puncte de referință (waypoint-uri) care descriu axul traseului. Sistemele bazate pe waypoint-uri oferă un control fin asupra formei circuitului și permit reutilizarea acelorași date atât pentru construcția vizuală a traseului, cât și pentru navigarea agenților AI [1].

Implementarea noastră utilizează un sistem unificat de waypoint-uri, configurate manual în editorul Unity, care definesc forma traseului și servesc simultan ca repere pentru navigarea vehiculelor AI.

3 Metodologie și Implementare

Proiectul este dezvoltat în Unity 6000.4.2f1 utilizând Universal Render Pipeline (URP) și New Input System. Arhitectura software este organizată modular, cu componente separate pentru fizica vehiculului, inteligența artificială, definirea traseului prin waypoint-uri și interfața utilizator.

3.1 Arhitectura Generală

Structura proiectului urmează o organizare standard Unity, cu următoarele module principale:

- **Scripts/** – conține logica jucătorului (`SimplePlayerCar`, `CameraFollow`)
- **Scripts/OpponentCarAI/** – conține AI-ul oponentilor (`OpponentCar`, `OpponentCarWaypoints`, `Waypoint`, `BoostPad`, `SpeedBreakers`)
- **Scripts/UI/** – conține sistemele de interfață (`LapSystem`, `MissionEndUI`, `PauseMenu`, `RaceCountdown`)
- **Scripts/Editor/** – instrumente pentru editorul Unity (`WaypointEditor`, `WaypointManagerWindow`)
- **Editor/** – scripturi de configurare automată (`AutoSetupScene`)

3.2 Fizica Vehiculului Jucătorului

Componenta centrală a proiectului este clasa SimplePlayerCar, care implementează un model complet de dinamică a vehiculului. Vehiculul este modelat ca un corp rigid (Rigidbody) cu masă de 1500 kg, echipat cu patru WheelCollider-e care simulează suspensia, fricțiunea și forțele generate de contact cu suprafața drumului.

3.2.1 Modelul Motorului și Curba de Cuplu

Cuplul motorului este calculat în funcție de viteza curentă a vehiculului, folosind o curbă adaptivă:

$$T_{curve} = 1 - \frac{v_{kmh}}{v_{max}} \quad (1)$$

Pentru viteze mici ($v < 60$ km/h), se aplică un factor de boost pentru accelerare rapidă:

$$T_{curve}^* = \text{lerp} \left(2.5, T_{curve}, \frac{v_{kmh}}{60} \right) \quad (2)$$

Cuplul efectiv per roată motrică se calculează ca:

$$T_{wheel} = \frac{I_{move} \cdot T_{max} \cdot T_{curve}^*}{N_{motor}} \quad (3)$$

unde I_{move} este inputul jucătorului ($[-1, 1]$), $T_{max} = 8000$ Nm este cuplul maxim, iar $N_{motor} = 4$ (în modul AWD) sau 2.

3.2.2 Controlul Tracțiunii

Sistemul de control al tracțiunii previne patinarea roților prin monitorizarea diferenței dintre viteza perifericii a roții și viteza vehiculului:

$$v_{wheel} = |RPM \cdot r \cdot 2\pi \cdot 60 / 1000| \quad (4)$$

Dacă $v_{wheel} > v_{vehicle} + \Delta_{slip}$ (unde $\Delta_{slip} = 15$ km/h), cuplul motorului pentru acea roată este setat la zero.

3.2.3 Downforce și Asistență Laterală

Downforce-ul simulat crește aderența la viteze mari prin aplicarea unei forțe orientate în jos, proporțională cu viteza:

$$\vec{F}_{down} = -\hat{u}_{car} \cdot k_{down} \cdot |\vec{v}| \quad (5)$$

unde $k_{down} = 50$ este coeficientul de downforce. Asistența la aderența laterală contracarează alunecarea laterală:

$$\vec{F}_{grip} = -\hat{r}_{car} \cdot v_{lateral} \cdot m \cdot k_{grip} \quad (6)$$

unde $k_{grip} = 5$ și $v_{lateral} = \vec{v} \cdot \hat{r}_{car}$.

3.2.4 Direcția Adaptivă

Unghiul maxim de virare scade progresiv cu viteza pentru a menține stabilitatea:

$$\alpha_{max} = \text{lerp} \left(\alpha_{high}, \alpha_{low}, \frac{v_{kmh}}{v_{dropoff}} \right) \quad (7)$$

cu $\alpha_{high} = 30$, $\alpha_{low} = 8$ și $v_{dropoff} = 120$ km/h. Trecerea la unghiul dorit se realizează prin interpolare liniară cu un factor de netezire.

3.2.5 Coliziunea cu Pereții

Sistemul de gestionare a coliziunilor cu pereții (OnCollisionEnter) calculează unghiul de impact folosind normala de contact proiectată pe planul orizontal. Dacă unghiul este sub un prag configurat (30), vehiculul alunecă de-a lungul peretelui cu o penalizare de viteză proporțională cu unghiul:

$$s_{mult} = \text{lerp} \left(s_{base}, 0, \frac{\theta_{impact}}{\theta_{threshold}} \right) \quad (8)$$

3.3 Inteligența Artificială a Oponenților

3.3.1 Navigarea Bazată pe Waypoint-uri

Sistemul AI este compus din trei componente principale:

1. **Waypoint** – definește un punct de referință pe traseu, cu legături către waypoint-urile precedent și următor (listă dublu înălțuită). Fiecare waypoint are o lățime configurabilă ($[0, 5]$ metri), iar poziția efectivă returnată este aleatorizată pe lățimea waypoint-ului:

$$\vec{p} = \text{lerp}(\vec{p}_{min}, \vec{p}_{max}, r), \quad r \sim U(0, 1) \quad (9)$$

2. **OpponentCarWaypoints** – controlează navigarea, actualizând destinația vehiculului AI atunci când waypoint-ul curent este atins.
3. **OpponentCar** – implementează locomoția efectivă prin rotirea progresivă spre destinație și ajustarea vitezei.

3.3.2 Locomoția Vehiculului AI

Vehiculul AI se rotește progresiv spre destinație folosind Quaternion.RotateTowards:

$$q_{new} = \text{RotateTowards}(q_{current}, q_{target}, \omega \cdot \Delta t) \quad (10)$$

cu $\omega = 30/s$. Viteza este ajustată prin accelerare liniară:

$$v_{new} = \text{MoveTowards}(v_{current}, v_{max}, a \cdot \Delta t) \quad (11)$$

Velocitatea finală este interpolată lin pentru a evita discontinuitățile care ar putea destabiliza fizica:

$$\vec{v}_{rb} = \text{lerp}(\vec{v}_{rb}, \vec{v}_{target}, 10 \cdot \Delta t) \quad (12)$$

3.3.3 Zone de Boost și Încetinire

Traseul conține zone speciale implementate ca trigger-e:

- **BoostPad:** Crește accelerația $([4, 5])$ și viteza curentă $([35, 41]$ unități) la intrarea în zonă.
- **SpeedBreakers:** Reduce accelerația $([0.5, 1])$ și viteza $([25, 28]$ unități) temporar, cu revenire la normal după o durată configurabilă (implicit 3 secunde) prin corutină.

3.3.4 Sistemul de Respawn

Vehiculul AI dispune de un mecanism de respawn care se activează când rămâne blocat (viteză < 2 unități/s) pentru mai mult de 10 secunde. La respawn, vehiculul este teleportat la destinația curentă, cu orientarea calculată pe baza direcției waypoint-ului următor.

3.4 Definirea Traseului prin Waypoint-uri

Traseul de curse este construit integral prin sistemul de waypoint-uri descris în secțiunea anterioară. Fiecare waypoint este un GameObject plasat manual în scenă, conectat cu cel precedent și cu cel următor printr-o listă dublu înlănțuită, astfel încât secvența de waypoint-uri descrie axul traseului.

Pentru a facilita editarea, scriptul WaypointEditor desenează în Scene View, prin API-ul Gizmos, conexiunile dintre waypoint-uri (linii verzi către succesor și roșii către predecesor) și lățimea configurată a fiecărui punct. Fereastra de editor WaypointManagerWindow oferă operații de creare, ștergere și reordonare a waypoint-urilor.

Aceeași secvență de waypoint-uri este utilizată simultan pentru două scopuri:

- ca traiectorie de referință pentru navigarea vehiculelor AI (vezi Secțiunea anterioară);
- ca structură geometrică pentru poziționarea segmentelor de drum și a pereților, plasate manual de-a lungul waypoint-urilor în editorul Unity.

Această abordare unificată evită duplicarea datelor între reprezentarea vizuală a traseului și logica de navigare a AI-ului, asigurând că oponenții urmează exact forma circuitului.

3.5 Sistemul Camerei

Componenta CameraFollow implementează o cameră de tip third-person orbitală. Camera urmărește vehiculul jucătorului cu un offset configurabil (distanță = 6 m, înălțime = 1.5 m) și permite controlul prin mouse cu sensibilitate ajustabilă. Poziția este netezită prin interpolare liniară (Vector3.Lerp).

3.6 Interfața Utilizator

3.6.1 Numărătoarea Inversă

Componenta RaceCountdown implementează un cronometru de pornire (3-2-1-GO!) folosind corutine. Pe durata numărării, Time.timeScale este setat la 0, oprind efectiv simularea fizicii.

3.6.2 Sistemul de Tururi

Clasa LapSystem utilizează un trigger collider plasat pe linia de start/finish pentru a detecta trecerea vehiculelor. La completarea unui tur, se verifică dacă cursa s-a încheiat (număr maxim de tururi atins).

3.6.3 Meniul de Pauză și Ecranul de Final

PauseMenu permite oprirea/reluarea jocului (Escape), restart și întoarcerea la meniul principal. MissionEndUI afișează rezultatul cursei (câștig/pierdere) și oferă opțiunea de întoarcere la meniul principal.

4 Rezultate

Proiectul a fost testat în Unity 6000.4.2f1, pe un circuit construit prin sistemul de waypoint-uri descris în secțiunea de metodologie.

4.1 Evaluarea Fizicii Vehiculului

Parametrii vehiculului jucătorului și efectele lor sunt sumarizate în Tabelul 1.

Parametru	Valoare	Efect
Masă vehicul	1500 kg	Stabilitate
Cuplu maxim motor	8000 Nm	Accelerare
Viteză maximă	320 km/h	Limita superioară
Forță de frânare	15000 Nm	Frânare eficientă
Coeficient downforce	50	Aderență la viteză
Asistență laterală	5	Anti-derapaj
Fricțiune normală	3.5	Grip anvelope

Tabela 1: Parametrii fizici ai vehiculului jucătorului.

Testele au demonstrat:

- **Accelerare progresivă:** Factorul de boost ($2.5\times$) la viteze mici asigură o pornire rapidă.
- **Stabilitate la viteză mare:** Downforce-ul și asistența laterală elimină oscilațiile.
- **Controlul tracțiunii:** Limita de 15 km/h previne patinarea roților.
- **Coasting natural:** Drag-ul aerodinamic oferă decelerare graduală.

4.2 Evaluarea Inteligenței Artificiale

Criteriu	Rezultat
Navigare completă traseu	Da
Evitarea blocajelor (respawn)	Activ (10s)
Variabilitate traiectorie	Aleatorizare pe lățime
Adaptare zone boost/breakers	Funcțională

Tabela 2: Evaluarea calitativă a sistemului AI.

4.3 Definirea Traseului prin Waypoint-uri

Sistemul de waypoint-uri permite construirea unor trasee de forme arbitrare direct din editorul Unity, cu vizualizare în Scene View a conexiunilor și a lățimii fiecărui punct. Aceeași secvență de waypoint-uri este folosită atât pentru poziționarea segmentelor de drum și a pereților, cât și pentru navigarea AI, eliminând duplicarea datelor între reprezentarea vizuală și logica de navigare.

4.4 Sistemele de Joc

Numărătoarea inversă, sistemul de tururi, meniul de pauză și ecranul de final funcționează conform așteptărilor, cu gestionare corectă a `Time.timeScale` și a cursorului mouse.

5 Direcții Viitoare

Proiectul actual oferă o bază solidă care poate fi extinsă în mai multe direcții:

- **AI avansat:** Înlocuirea navigării pe waypoint-uri cu tehnici de reinforcement learning (ex. PPO, SAC) care ar permite oponentilor să învețe strategii de depășire și apărare a poziției.
- **Generare procedurală a traseelor:** Adăugarea unui algoritm de generare automată a waypoint-urilor pentru a crea trasee variate (serpentine, intersecții, diferențe de altitudine), eventual prin curbe Bézier sau spline-uri interpolate peste waypoint-uri.
- **Multiplayer:** Adăugarea suportului pentru joc în rețea (multiplayer online) utilizând Unity Netcode for GameObjects sau Mirror.

- **Sistem de tuning:** Implementarea unui meniu de personalizare a vehiculului (motor, suspensie, anvelope) care să modifice parametrii fizicii în timp real.
- **Efecte vizuale și audio:** Adăugarea particulelor (fum la derapaj, scântei la coliziune), urmelor de anvelope pe asfalt și a unui motor audio cu pitch variabil.
- **Sistem de clasament:** Implementarea unui clasament în timp real care să indice pozițiile vehiculelor pe baza distanței parcurse pe traseu.

6 Concluzii

Această lucrare a prezentat proiectarea și implementarea unui joc de curse auto 3D în Unity, demonstrând aplicarea practică a conceptelor de proiectare avansată a sistemelor software. Principalele contribuții ale proiectului sunt:

1. **Un model de fizică vehiculară realist** bazat pe WheelCollider-e Unity, augmentat cu sisteme de control al tracțiunii, downforce simulat, asistență la aderența laterală și gestionare inteligentă a coliziunilor cu pereții. Aceste sisteme cooperează pentru a oferi o experiență de conducere stabilă și responsabilă.
2. **Un sistem AI funcțional** bazat pe navigarea prin waypoint-uri, cu variabilitate introdusă prin aleatorizarea poziției pe lățimea waypoint-ului, zone de boost/încetinire și mecanism de recuperare automată din blocaje.
3. **Definirea traseului prin waypoint-uri** configurate în editorul Unity, soluție care unifică reprezentarea vizuală a circuitului cu datele de navigare ale AI-ului și permite forme arbitrare de traseu.
4. **Sisteme complete de game flow** – numărătoare inversă, contorizare tururi, pauză și ecran de final – care formează o experiență de joc completă.

Proiectul ilustrează importanța arhitecturii modulare în dezvoltarea de software complex: fiecare subsistem (fizică, AI, traseu, UI) este implementat ca o componentă independentă, ușor de testat și de extins.

7 Limitări

Proiectul prezintă următoarele limitări:

- **Fizica AI simplificată:** Vehiculele AI nu utilizează WheelCollider-e pentru locomotie (doar pentru vizualizarea rotației roților), ci controlează direct viteza RigidBody-ului. Aceasta poate duce la comportamente nerealiste la coliziuni între vehicule.
- **Traseu static:** Traseul este construit prin waypoint-uri plasate manual în editor și are o formă fixă, fără variabilitate în topologia circuitului între sesiuni de joc.
- **Lipsa sistemului de clasament:** Nu există un clasament în timp real care să indice pozițiile relative ale vehiculelor pe parcursul cursei.
- **Navigare AI limitată:** Oponenții urmează strict waypoint-urile fără comportamente de evitare a obstacolelor sau de depășire a altor vehicule.
- **Lipsa efectelor vizuale avansate:** Nu sunt implementate particule, urme de anvelope sau efecte sonore specifice (motor, derapaj, coliziune).

Bibliografie

- [1] Rabin, S. (2009). *Introduction to Game Development*. Charles River Media.
- [2] Pacejka, H. B. (2012). *Tire and Vehicle Dynamics* (3rd ed.). Butterworth-Heinemann.
- [3] Unity Technologies. (2024). *Unity Manual: Wheel Collider*. <https://docs.unity3d.com/Manual/class-WheelCollider.html>. Accesat: Mai 2026.
- [4] Shaker, N., Togelius, J., & Nelson, M. J. (2016). *Procedural Content Generation in Games*. Springer.
- [5] Kiran, B. R., Sobh, I., Talpaert, V., Mannion, P., Sallab, A. A. A., Yogamani, S., & Pérez, P. (2021). Deep Reinforcement Learning for Autonomous Driving: A Survey. *IEEE Transactions on Intelligent Transportation Systems*, 22(8), 4909–4926.

- [6] Gregory, J. (2018). *Game Engine Architecture* (3rd ed.). CRC Press.